

Explain It!

Ask Anything. See It Clearly.

Why We Generate, Check, and Compose High-School Lessons in Stages

Technical Product White Paper, Version 3.1

Meiyu Shen

Yifan Yan

Jiachen Yu

Jiaxing Senior High School (BC Offshore Program)

August 2026

Abstract

We present the Explain It! system, a deployable lesson studio that turns a Grades 9–12 question into a structured explanation, source-grounded notes, a quiz, a connected concept map, and an optional narrated video. We ask a practical design question: if generation is probabilistic and media calls are expensive, where should a small educational system check its work? We answer with a staged pipeline. Gate 1 screens the explanation before research and media spending. Gate 2 checks seven image frames with parallel pixel diagnostics and, only when those checks pass, one semantic contact-sheet review. We generate video by composing inspected still images, captions, transitions, and narration rather than by calling a direct text-to-video model. The design follows established ideas from multimedia learning, cognitive-load research, retrieval practice, feedback research, and risk-based AI evaluation, while remaining explicit that automated gates do not prove factual correctness (Mayer, 2009; Sweller, 1988; Roediger and Karpicke, 2006; Hattie and Timperley, 2007; Autio et al., 2024).

This Version 3.1 paper documents the deployed system available in August 2026. It includes the optional OpenAI media key shown when DeepSeek is selected for text, the directional and interactive concept neighborhood, live per-frame progress, and stage-aware study tips. We derive the gate equations using high-school algebra, probability, weighted averages, inequalities, and directed graphs. Proofs establish what the formulas do guarantee—bounded scores, noncompensatory accuracy, all-frame enforcement, expected-cost ordering, parallel-time bounds, conserved scene duration, and monotone progress—and state what they cannot guarantee. The application is available at <https://teachhighschool.streamlit.app/>; source code is available at <https://github.com/allanzhang721/AiLessonStudio>.

1 Introduction

A fluent answer is not automatically a good lesson. It may omit a condition, jump over a reasoning step, use language above the student’s grade, or present a polished but incorrect diagram. These risks matter for high-school learners because a student may not yet have enough background knowledge to notice the error. At the same time, checking everything with another large model can double cost and latency without creating independent evidence.

We built Explain It! around this tension. We do not ask one model call to write an answer, find sources, draw figures, narrate them, and judge the result. We separate these responsibilities. The student first receives an explanation that has passed a local and semantic review. Research notes are generated separately so citations remain visible. The visual pipeline then creates seven inspectable frames. Only after those frames pass Gate 2 do we compose the final MP4.

The uploaded SLaTR manuscript influenced our writing habits: we begin from the uncertainty, define each object before using it, explain the mechanism next to the corresponding diagram, and use “we”

when describing choices made by the project team. We do not reproduce its technical claims or wording. This document is a project white paper rather than a peer-reviewed research article.

We make eight contributions:

1. We provide a complete public application rather than a notebook-only prototype.
2. We separate text capability from media capability. OpenAI can reuse one key for both; DeepSeek remains text-only but can be paired, only when the learner chooses, with a separate OpenAI image-and-narration key.
3. We implement a local-first Gate 1 that combines transparent structural checks with a compact five-dimension semantic rubric and at most one repair.
4. We implement a local-first Gate 2 that checks seven frames in parallel and makes at most one semantic vision call.
5. We compose video from approved still frames and audio, which makes every teaching step observable and regenerable.
6. We expose evidence, misconceptions, practice, quality records, and related-topic sources instead of returning one uninterrupted block of generated text.
7. We turn the concept map into a directional study tool with relationship filters, focused-topic navigation, challenges, curated links, and self-reported connection progress.
8. We expose honest stage and per-frame progress together with short retrieval, source-checking, diagram-reading, and study tips during long calls.

2 Scope, Claims, and Intended Use

We target short explanatory lessons for students in Grades 9–12. The system can support self-study, teacher preparation, and exploration of a difficult concept. It is not a teacher, textbook, examination authority, or diagnostic instrument. A passing gate means that the artifact met the implemented checks; it does not mean that the artifact is certainly correct.

Table 1 separates claims we can support from claims we do not make.

Supported project claim	Claim outside the evidence
The code runs two staged quality gates and records their decisions.	A passing gate guarantees scientific truth or classroom safety.
The saved experiments compare named methods on their recorded datasets.	Offline accuracy equals real-world lesson correctness.
The image-composition path exposes seven inspectable frames.	Seven frames are optimal for every subject or learner.
The quiz provides immediate item feedback and review suggestions.	One five-item quiz estimates long-term mastery.
Visitor keys are not intentionally written to project files or manifests.	The application controls a third-party provider after a request is sent.

Table 1: The paper treats quality gates as risk-reduction controls, not proofs.

3 Evidence Behind the Learning Design

3.1 Why we divide a lesson into visible parts

Cognitive-load research distinguishes useful reasoning from processing consumed by the presentation itself. Sweller’s early analysis argues that demanding problem-solving procedures can consume capacity that would otherwise support schema acquisition (Sweller, 1988). Multimedia-learning research likewise

reports benefits from removing irrelevant material, signaling structure, and coordinating words with graphics (Mayer, 2009; Fiorella and Mayer, 2021). We translate these findings into conservative product choices: a focused explanation, explicit key ideas, a worked example, a reserved caption band, a limited number of visual steps, and restrained motion.

These sources do not prove that our interface improves learning. They explain why the interface is structured for inspection and low extraneous load. A classroom evaluation is still required.

3.2 Why we include retrieval and feedback

Retrieval practice can improve delayed retention compared with repeated study, although the effect depends on the task and context (Roediger and Karpicke, 2006). Feedback can be powerful, but its effect depends on what it says and how it is used (Hattie and Timperley, 2007). We therefore include five short questions, answer explanations, and a list of missed concepts. We do not turn one quiz into an opaque mastery score.

3.3 Why we treat generated content as managed risk

Large language models create opportunities for education but also introduce factual, pedagogical, privacy, and overreliance risks (Kasneci et al., 2023). NIST frames generative-AI assurance as an ongoing process of mapping, measuring, and managing risk rather than as a one-time certificate (Autio et al., 2024). Our gates follow the same practical direction: define narrow checks, record what ran, block expensive downstream work when an upstream artifact fails, and preserve a human role.

4 System Overview

Let a request be

$$x = (q, s, g, \ell, p_T, m_T, k_T, p_I, m_I, k_I, u), \quad (1)$$

where q is the question, s the subject, g the grade, ℓ the language, (p_T, m_T, k_T) the text provider, model, and session key, (p_I, m_I, k_I) the optional media configuration, and u the user options such as research and video. When OpenAI supplies text, $k_I = k_T$; when DeepSeek supplies text, k_I is empty unless the learner explicitly opens the optional media connection. The output is a lesson bundle

$$y = (e, r, z, G, F, a, v, h), \quad (2)$$

where e is the approved explanation, r the research report, z the quiz, G the concept graph, $F = (I_1, \dots, I_7)$ the frame sequence, a the narration, v the MP4, and h the quality history.

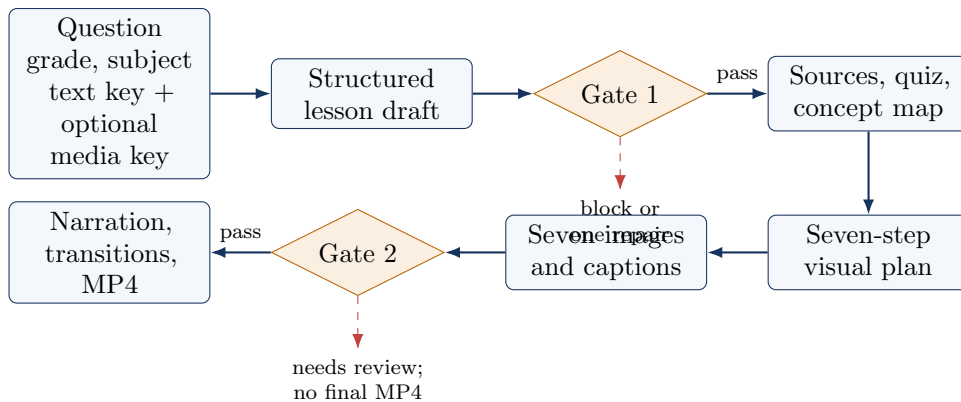


Figure 1: Production architecture. The ordering is also a spending policy: text is checked before image calls, and frames are checked before final composition.

The application stores explicit status rather than a single success flag. Gate 2 can be waiting, blocked by Gate 1, not applicable to a text-only lesson, incomplete because media failed, passed, or marked for review. This prevents a narration failure from erasing a completed visual audit.

5 Why We Use a Staged Cascade

Suppose a text check costs C_T , research costs C_R , images cost C_I , and narration plus composition costs C_V . Let p_1 be the fraction of drafts that pass Gate 1 and p_2 the fraction of frame sets that pass Gate 2. A simplified expected variable cost is

$$\mathbb{E}[C] = C_T + p_1(C_R + C_I) + p_1p_2C_V. \quad (3)$$

The equation uses multiplication for a simple reason: the video stage is reached only when both earlier events occur. If a draft fails, we do not buy images for it. If its images fail, we do not pretend the final video is approved.

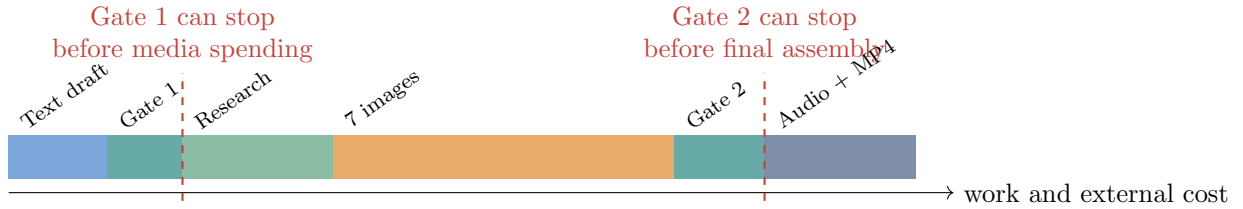


Figure 2: Qualitative cost path. Segment widths illustrate relative workflow weight; they are not provider prices. Actual prices and latency vary by model and date.

The cascade also improves diagnosis. A local readability warning, a factual rubric failure, a dark frame, and a sequence-continuity problem should not all appear as “generation failed.” Each belongs to a different stage and suggests a different correction.

6 Mathematical Design Questions

The project uses mathematics as a design language, not as decoration. Each formula answers a practical question that a high-school student can follow:

Question	Mathematical conclusion	Detailed proof
Can a weighted score leave the interval $[0, 1]$?	No, when every component is in $[0, 1]$, weights are nonnegative, and weights sum to one.	Proposition A.1
Can a high average hide a serious factual weakness?	Yes. A mandatory accuracy rule is therefore separate from the average.	Proposition A.3
Why must all seven frames pass?	A mean can hide one weak frame, and the chance of at least one defect rises with the number of frames.	Proposition A.6
Which check should run first?	For two independent screening stages, compare cost per rejection probability, c/r .	Theorem A.7
Why process frame diagnostics in parallel?	Parallel time is no greater than sequential time; with equal jobs and four workers, seven jobs need at least two waves.	Proposition A.9
Does timing allocation create or lose duration?	No. The scene-duration formula sums exactly to the chosen total and preserves the per-scene floor.	Proposition A.10

Table 2: The main mathematical questions behind the production design. The appendices use only algebra, averages, inequalities, elementary probability, and graphs.

A proof in this paper establishes a property of the *implemented formula under stated assumptions*. It does not prove that a model response is true, that a student learned, or that a chosen threshold is optimal. Those questions require observations and evaluation data.

7 Gate 1: Explanation Review

7.1 Local screen

The local screen checks four transparent conditions: the explanation contains 70–420 alphabetic words; it includes at least one important term from the question when such terms exist; and its mean sentence length is no more than 27 words. These values are engineering defaults for a focused lesson, not universal reading-level standards. The gate reports the violated condition instead of hiding it behind a classifier label.

7.2 Semantic rubric and bounded repair

A semantic reviewer scores accuracy, completeness, logical flow, grade fit, and clarity from 1 to 4. Accuracy must equal 4, and every other dimension must be at least 3. When the first review fails but returns a revision, the system allows one repair and one recheck. The maximum of two semantic calls bounds the loop.

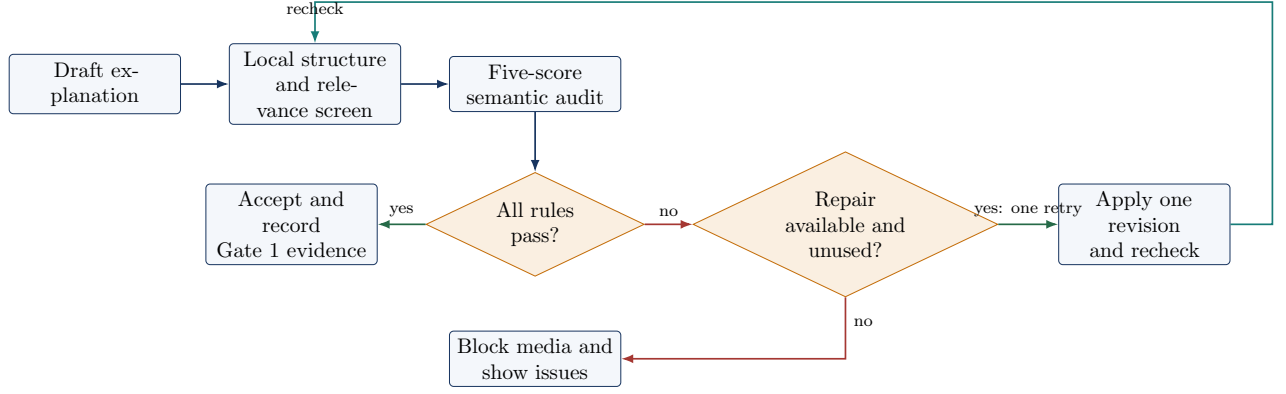


Figure 3: Gate 1 decision graph. The loop is bounded to prevent indefinite self-revision.

If the five scores are $s_d \in \{1, 2, 3, 4\}$, the displayed normalized score is

$$S_1 = \frac{1}{5} \sum_{d=1}^5 \frac{s_d}{4}. \quad (4)$$

A score of $S_1 = 0.90$ is not sufficient by itself. The logical pass rule still requires maximum accuracy and no failed local check. Appendix A gives the exact indicator form and examples.

7.3 Why the saved text classifier is not the production judge

The repository retains DistilBERT, BERT, and RoBERTa experiments. BERT introduced bidirectional transformer pretraining, RoBERTa revised its training recipe, and DistilBERT compressed a larger teacher model (Devlin et al., 2019; Liu et al., 2019; Sanh et al., 2019). Our saved models classify pedagogical error categories on the experiment dataset. That is useful offline evidence, but it is not the same task as verifying an arbitrary new explanation. We therefore report those models as baselines and use the narrower, inspectable cascade in production.

8 Gate 2: Visual Review

8.1 Parallel technical checks

Gate 2 receives seven ordered frames. For each frame, it measures width, height, mean grayscale brightness, grayscale contrast, edge variation, and variation in the lower caption band. Four worker threads process frames in parallel. Five normalized component scores then represent resolution, exposure, contrast, detail, and caption-band readability. Their weighted sum produces a technical frame score $t_i \in [0, 1]$.

The local stage uses a strict all-frame rule. Its mean must be adequate, and every required frame must meet the technical threshold 0.42. This is important because an average can hide one unusable teaching step. If any frame fails, the system stops before a vision-model call.

8.2 One semantic contact-sheet review

When the technical stage passes and a compatible vision client is available, the frames are resized into one ordered contact sheet. One call scores semantic accuracy, teaching alignment, sequence continuity, legibility, and visual load from 1 to 4. Semantic accuracy must equal 4 and all other dimensions must be at least 3. The combined display score is

$$S_2 = 0.35T + 0.65M, \quad (5)$$

where T is mean technical quality and M is normalized semantic quality. The larger semantic weight expresses an engineering priority: a sharp, bright image is still a poor teaching image if it communicates the wrong idea. The weights and thresholds were not learned from classroom outcomes and should be recalibrated when a suitable labeled dataset becomes available.

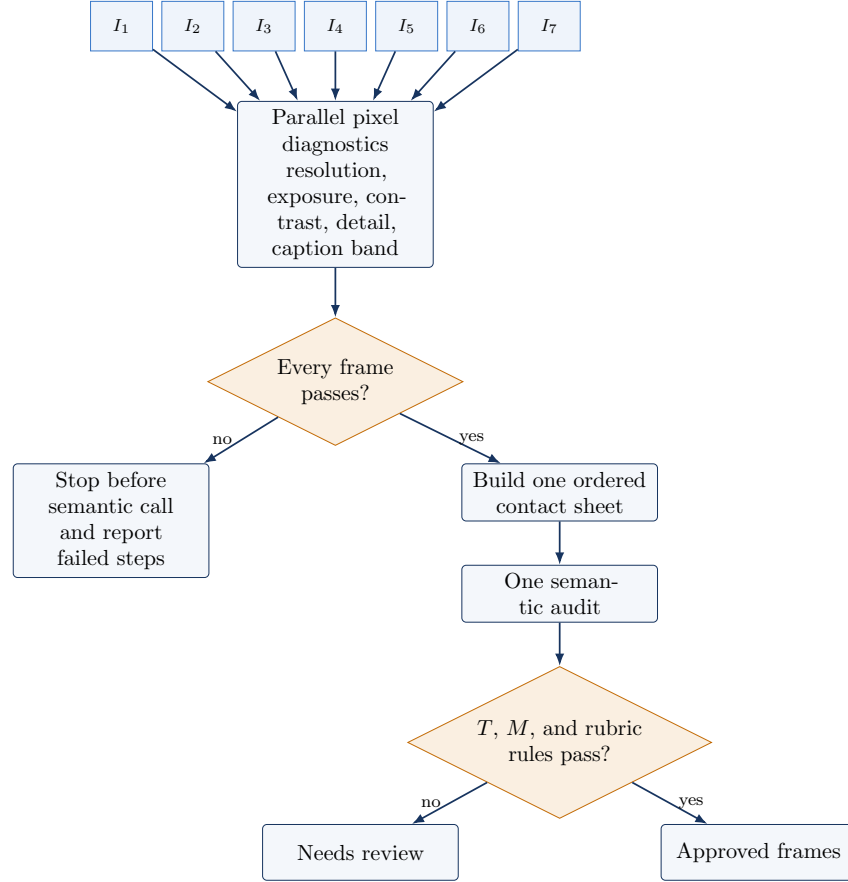


Figure 4: Gate 2 decision graph. Pixel diagnostics screen rendering failures; the contact sheet addresses meaning and continuity.

8.3 Why CLIP remains an offline comparison

CLIP learns an image–text representation through contrastive pretraining and can measure whether an image and caption are close in its embedding space (Radford et al., 2021). Our saved Gate 2 experiments compare cosine thresholding with linear and multilayer probes on aligned and mismatched Flickr8k pairs (Hodosh et al., 2013). This benchmark asks whether a caption matches an image. Production Gate 2 also asks whether labels are readable, exposure is usable, the sequence remains coherent, and the visual teaches the approved explanation. Because the tasks differ, we do not present the CLIP benchmark as a production certification.

9 From Approved Frames to Video

9.1 Why we removed direct text-to-video generation

We removed the direct video path for three reasons. First, a generated clip is harder to inspect at the level of individual teaching steps. Second, it can change objects, labels, or causal relations over time.

Third, its cost is difficult to control in a classroom workflow. The still-image route creates seven explicit checkpoints and allows visual-only regeneration without rewriting an approved explanation.

9.2 Storyboard contract

The planner produces a shared scene bible and seven ordered steps. Each step states its teaching goal, retained objects, additions, removals, labels, and misconceptions to avoid. The image prompt includes the same visual style, layout zones, typography, color assignments, and canonical explanation. A fixed lower band is reserved for captions. These constraints implement coherence and signaling as engineering rules rather than asking the image model to rediscover them for every frame (Fiorella and Mayer, 2021).

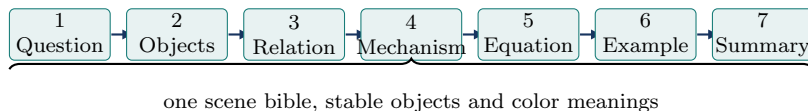


Figure 5: A seven-scene instructional timeline. Labels describe roles, not a mandatory subject template; the planner adapts them to the question.

9.3 Narration-aware timing and composition

The system estimates or measures narration length, gives every scene at least 3.2 seconds, and allocates remaining time by caption word count. Frames are fitted to a 16:9 canvas over a blurred background so content is not cropped. A restrained 1.8% Ken Burns movement alternates direction, and adjacent scenes cross-fade. The video is encoded as H.264 with CRF 18, browser-compatible yuv420p, and fast-start metadata. Narration is normalized toward -16 LUFS with a -1.5 dB true-peak ceiling when the FFmpeg filter is available. Appendix A gives the timing equation.

10 Student-Facing Lesson Structure

The generated bundle is rendered as six views:

1. **Lesson:** objective, explanation, key ideas, worked example, common mistake, and quick check.
2. **Concept map:** prerequisites, applications, contrasts, and next topics.
3. **Evidence and sources:** a grounded synthesis, source-by-source notes, and evidence limits.
4. **Practice:** five multiple-choice items with answer explanations and missed-concept guidance.
5. **Learning library:** deterministic links to established platforms rather than model-invented URLs.
6. **Quality report:** gate path, score, latency, issues, repairs, and model-call count.

Markdown and common LaTeX wrappers are normalized before Streamlit renders them. This prevents a useful equation such as $F = ma$ from appearing as raw control characters. The normalizer preserves paragraphs and lists instead of flattening the lesson into one line.

10.1 Useful waiting time rather than an empty spinner

Model and image calls can take long enough for a learner to disengage. The deployed interface therefore reports a stage label and a monotone progress fraction. During the seven-frame loop it reports when each illustration starts and finishes. The display does *not* claim to predict seconds remaining; provider queues and retries make such a prediction unreliable.

The same panel shows one short study action selected from a small code-controlled library. Drafting tips ask the learner to predict or recall; gate tips ask about units, assumptions, and counterexamples; research tips ask the learner to compare sources; frame tips ask the learner to follow arrows or explain visual change; composition tips recommend retrieval rather than passive replay. The selector uses only

stage, grade, subject text, and frame number. It does not infer emotion, attention, disability, or ability. Appendix A.11 proves that the displayed media progress never moves backward and explains the modulo selector used for tips.

10.2 Connected concepts as an interactive directed graph

The text model proposes five short topic connections. The program normalizes each relationship into one of five roles: prerequisite, application, contrast, next step, or related concept. It then discards any model-generated URL and attaches sources using deterministic subject mappings to platforms such as OpenStax, Khan Academy, GeoGebra, PhET, and MDN. This separation is important: the model proposes *what may be related*; code-controlled mappings decide *where a learner may click*.

The current view is not a static star diagram. A horizontal control filters learning roles. A focus selector and a *Next link* button move through topics. The focused topic shows why it matters, trusted links, and a relationship-specific challenge. A learner may mark “I can explain this link,” and a progress bar displays the number marked out of the total. This is a self-report and navigation aid, not a mastery diagnosis.

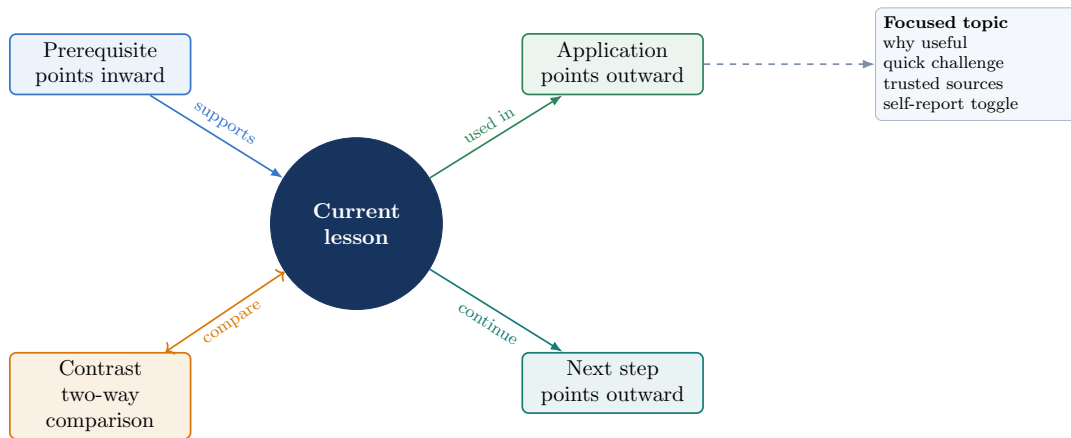


Figure 6: The deployed concept neighborhood encodes learning direction and connects the selected node to an interactive study panel. Filtering changes the displayed subgraph; it does not regenerate the lesson or make another model call.

Let the concept graph be $G = (V, E)$, where the current lesson and related topics are vertices and labeled relationships are directed edges. If a learner marks m of n related topics, the displayed self-report progress is m/n . Appendix A.12 proves its bounds and shows why adding one completed connection changes the bar by exactly $1/n$.

11 Offline Checker Experiments

The repository contains saved benchmark summaries for the earlier research checkers. Table 3 reports the values used by the application’s friendly experiment snapshot. Gate 1 and Gate 2 use different datasets and tasks, so their rows must not be ranked against one another.

Gate	Method	Accuracy	Precision	Recall	F1	AUROC
1	DistilBERT classifier	0.9280	0.9444	0.9280	0.9284	—
1	BERT classifier	0.9253	0.9314	0.9253	0.9249	—
1	RoBERTa classifier	0.9653	0.9667	0.9653	0.9655	—
2	CLIP cosine threshold	0.9628	0.9474	0.9800	0.9634	0.9922
2	CLIP linear probe	0.8756	0.8301	0.9444	0.8836	0.9555
2	CLIP MLP probe	0.9517	0.9462	0.9578	0.9520	0.9885

Table 3: Saved offline metrics. Gate 1 classifies pedagogical error types. Gate 2 classifies aligned versus mismatched Flickr8k pairs. “—” means AUROC was not retained in the summarized multiclass result.

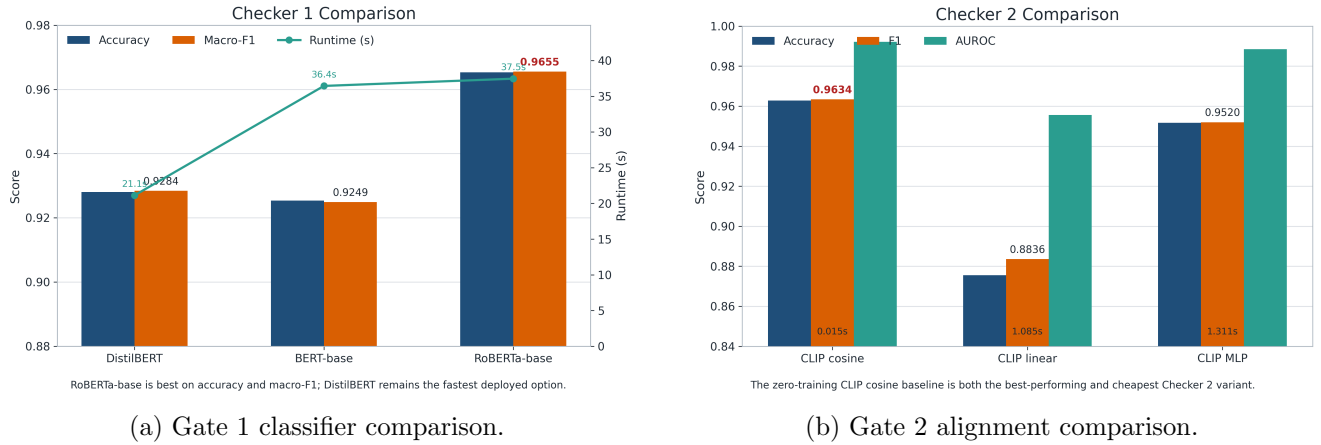


Figure 7: Offline experiment figures generated from repository records. These plots do not show live request latency or learning outcomes.

The RoBERTa classifier has the highest saved Gate 1 F1. The untrained CLIP cosine threshold has the highest saved Gate 2 F1 and AUROC. This result illustrates why “trained” is not itself a deployment metric. We also consider task match, startup weight, latency, interpretability, and whether a baseline measures the product failure we need to catch.

11.1 Metric interpretation

Accuracy asks what fraction of all predictions were correct. Precision asks, among examples predicted positive, how many were positive. Recall asks, among all positive examples, how many were found. F1 is the harmonic mean of precision and recall. AUROC summarizes ranking across thresholds. Appendix A defines each quantity and gives a small confusion-matrix example. For imbalanced classes, accuracy alone can be misleading; this is why we retain multiple metrics (Fawcett, 2006).

12 Reproducible Local Setup

12.1 Requirements

The production target is Python 3.12. The minimal application dependencies are OpenAI’s Python SDK, Pillow, ImageIO, ImageIO-FFmpeg, Streamlit, Requests, and Graphviz. A local Graphviz executable may also be required for graph rendering. Research and training dependencies are isolated from the website requirements.

12.2 Installation

```
git clone https://github.com/allanzhang721/AiLessonStudio.git
cd AiLessonStudio
python -m venv .venv
# Windows PowerShell
.venv\Scripts\Activate.ps1
# macOS or Linux: source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pytest -q
python -m streamlit run streamlit_app.py
```

The app opens before a key is entered, but generation remains locked until the required fields are present. The first password field is always the text key. With OpenAI selected for text, that same key can be reused for research, images, visual review, and narration. With DeepSeek selected, the interface states that DeepSeek is text-only in this application. The learner may stop there and generate a written lesson, or open *Add image API (optional)*. Only then does a second OpenAI image-key field appear; that optional key supplies images, the semantic visual audit, narration, and MP4 composition while DeepSeek continues to write the lesson.

12.3 Directory map

```
streamlit_app.py # production entrypoint
app_v2.py # complete Streamlit interface
pipeline/
  lesson_service.py # bundle generation, research, sources
  quality_gates.py # Gate 1
  frame_checker.py # Gate 2
  planner.py # seven-scene plan
  image_pipeline.py # frame generation
  video_pipeline.py # timing, motion, audio, MP4
  markdown_render.py # Markdown and LaTeX normalization
  gate_benchmarks.py # small offline metric snapshot
tests/ # deployment and regression tests
requirements.txt # production dependencies
requirements-research.txt # optional experiment dependencies
```

12.4 Test protocol

The production subset contains 33 executable tests at this paper version. It covers frame scoring and short-circuit behavior, explanation repair, provider routing, the conditional second-key Streamlit flow, concept relationship direction, progress callbacks, waiting-tip selection, source annotation conversion, schema normalization, Markdown and LaTeX handling, fail-fast plan validation, student-analysis helpers, narration arguments, audio-aware timing, and real H.264 encoding. Six additional research-only tests depend on experiment modules and data that are intentionally not shipped with the lightweight deployment checkout. The production command is:

```
python -m pytest -q tests \
  --ignore=tests/test_checker1_research.py \
  --ignore=tests/test_checker2_research.py
```

A passing unit suite establishes that the code meets the tested contracts. It does not establish educational effectiveness. A publication evaluation should additionally include expert factual review, student comprehension measures, accessibility review, cost records, and failure analysis.

13 Deployment, Privacy, and Operational Boundaries

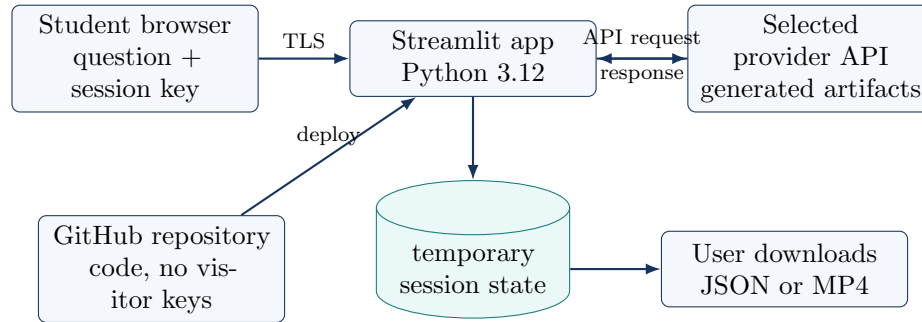


Figure 8: Deployment and data path. The project does not intentionally persist visitor keys; external providers still receive the requests authorized by those keys.

The public deployment uses the [AiLessonStudio repository](#). Its production coordinates are branch `agent/streamlit-production`, endpoint `streamlit_app.py`, and Python 3.12. Streamlit Community Cloud installs dependencies from the repository and supports configuration of Python and secrets through its deployment settings (Streamlit, 2026a,b). The current visitor flow does not require a repository secret because the visitor provides a session text key and, only when pairing a text-only provider with media, an optional session image key.

The password-style field reduces accidental display but is not a complete security boundary. The key is passed to the chosen provider client. Students should use restricted keys, set provider-side budgets, avoid entering personal information in prompts, and revoke a key they believe was exposed. School deployments should perform their own privacy, consent, procurement, and retention review.

14 Source and Figure Provenance

Table 4 records where each kind of paper evidence comes from. This is important because a system diagram, an experiment plot, and an external research claim have different evidentiary status.

Artifact	Source	Interpretation
Figures 1–8	Drawn in TikZ from the production source code and deployment configuration	Explanatory system diagrams; not experimental observations
Figure 7	Repository plotting outputs and saved benchmark values	Offline method comparison on recorded tasks
Table 3	<code>pipeline/gate_benchmarks.py</code>	Distilled values used by the public dashboard
Gate, timing, progress, and graph equations	<code>quality_gates.py</code> , <code>frame_checker.py</code> , <code>pipeline.py</code> , <code>video_pipeline.py</code> , and <code>lesson_service.py</code>	Exact implementation or direct algebraic consequence as of the paper version
Learning rationale	Cited books, peer-reviewed articles, and public standards	Motivation for design choices, not a direct evaluation of this app
Setup instructions	Repository files and official Streamlit documentation	Reproducibility and deployment procedure

Table 4: Evidence and figure provenance. No generated stock artwork is used in the technical diagrams.

15 Limitations and Threats to Validity

Shared-model bias. A generator and semantic reviewer from related model families can share mistakes. The second call is a structured audit, not independent expert replication.

Proxy measurements. Brightness, contrast, and edge variation detect technical defects but do not establish scientific correctness. Likewise, an image-caption benchmark does not measure the learning value of a seven-frame lesson.

Threshold selection. The production thresholds are transparent engineering defaults. They have not been calibrated on a representative sample of teacher-rated generated lessons. Appendix C lists each threshold so it can be challenged and replaced.

Dataset mismatch. Gate 1 offline models classify error categories. Gate 2 offline models use Flickr8k alignment. Neither dataset estimates the probability that a new lesson improves high-school comprehension.

Provider dependence. Model behavior, availability, price, and policy can change. The paper records product logic rather than promising a permanent provider capability.

Accessibility and language. The interface supports multiple output languages, but the structural word rules are based on English alphabetic tokens. Caption readability metrics do not replace screen-reader, color-contrast, hearing-access, or multilingual review.

No classroom trial. We have not yet reported a randomized or controlled student study. The correct next publication step is an evaluation with teachers and learners, not stronger language about the current benchmarks.

16 Recommended Publication Evaluation

We recommend a preregistered evaluation with three layers.

1. **Artifact review:** subject teachers rate factual accuracy, missing conditions, grade appropriateness, and diagram correctness for a stratified set of questions.
2. **System review:** measure latency, provider cost, gate pass rate, repair rate, false blocks, missed errors, and failure recovery.
3. **Learner study:** compare an approved text lesson, an approved text-plus-video lesson, and a text-book/control condition using immediate and delayed questions. Record prior knowledge and avoid claiming personalization from one quiz.

For gate calibration, teachers should label individual failure types and final usability independently. We can then choose thresholds on a validation split and report confidence intervals on a held-out test split. Precision and recall should be reported for the harmful-artifact class because missing a bad lesson and unnecessarily blocking a good lesson have different costs.

17 Conclusion

We built Explain It! as a staged lesson studio because generated educational artifacts fail in different ways and because later media stages are costly. Gate 1 checks the explanation before research and images. Gate 2 checks every frame before final composition. The video path keeps seven inspectable images, stable captions, controlled timing, per-frame progress, and a recoverable intermediate state. The current app also supports a DeepSeek text lesson with an optional OpenAI media connection, a directional concept neighborhood with study challenges and self-report progress, cited notes, misconceptions, practice, regeneration, and a visible quality report.

The central lesson is not that more checking guarantees truth. It is that narrow checks, placed at the right boundaries, make failures cheaper to detect and easier to explain. The offline classifiers remain useful comparisons. The live gates remain engineering controls. Teacher and student evaluation remains necessary. Keeping these statements separate makes the project easier to reproduce, criticize, and improve.

A Mathematical Appendix: Why the Gates Have This Form

This appendix is written for a student who knows percentages, averages, algebra, graphs, and elementary probability. We use proofs to establish properties of the code. We do not use a formula as evidence that the generated lesson is factually correct.

A.1 Definitions and assumptions

A score is *normalized* when it lies between 0 and 1. A normalized score of 0.8 means 80% of the available score; it does not automatically mean an 80% probability of correctness. An indicator

$$\mathbf{1}[A] = \begin{cases} 1, & A \text{ is true,} \\ 0, & A \text{ is false} \end{cases}$$

turns a logical rule into arithmetic. Products of indicators represent “and”: the product is 1 only when every factor is 1.

Throughout the appendix, weights are nonnegative. Probabilistic examples state when independence is assumed. Costs are illustrative units unless a provider price sheet and date are supplied.

A.2 Why normalized weighted averages stay normalized

Proposition A.1 (Convex-score bound). *Let $x_1, \dots, x_d \in [0, 1]$ and let $w_1, \dots, w_d \geq 0$ with $\sum_{j=1}^d w_j = 1$. Then*

$$S = \sum_{j=1}^d w_j x_j$$

also satisfies $0 \leq S \leq 1$.

Proof. Because each w_j and x_j is nonnegative, every product $w_j x_j$ is nonnegative, so $S \geq 0$. Also $x_j \leq 1$, hence $w_j x_j \leq w_j$. Adding these inequalities gives

$$S = \sum_j w_j x_j \leq \sum_j w_j = 1.$$

Therefore $S \in [0, 1]$. □

This simple result explains why Gate 2 weights sum to one. It also makes the displayed percentage interpretable without extra clipping after aggregation.

Proposition A.2 (Sensitivity of a weighted score). *If one component x_k increases by Δ while all other components stay fixed, then S increases by exactly $w_k \Delta$.*

Proof. The new score is

$$S' = \sum_{j \neq k} w_j x_j + w_k(x_k + \Delta) = S + w_k \Delta.$$

Subtracting S gives $S' - S = w_k \Delta$. □

Thus a weight is also a sensitivity coefficient. In Gate 2, a 0.10 improvement in semantic score changes the combined score by $0.65(0.10) = 0.065$, whereas the same technical improvement changes it by $0.35(0.10) = 0.035$.

A.3 Gate 1: an average plus mandatory conditions

Let

$$\mathbf{s} = (s_a, s_c, s_l, s_g, s_r), \quad s_j \in \{1, 2, 3, 4\},$$

represent accuracy, completeness, logical flow, grade fit, and readability. The displayed score is

$$S_1 = \frac{s_a + s_c + s_l + s_g + s_r}{20}. \tag{6}$$

Let J be the number of local structural issues and $b \in \{0, 1\}$ the reviewer's explicit decision. The production pass rule is

$$P_1 = \mathbf{1}[J = 0] \mathbf{1}[s_a = 4] \mathbf{1}[\min(s_c, s_l, s_g, s_r) \geq 3] \mathbf{1}[b = 1]. \tag{7}$$

Proposition A.3 (A high mean cannot protect a mandatory dimension). *For the five Gate 1 dimensions, a lesson can score at least 0.95 while failing the required accuracy condition.*

Proof. Choose $(s_a, s_c, s_l, s_g, s_r) = (3, 4, 4, 4, 4)$. Then

$$S_1 = \frac{3 + 4 + 4 + 4 + 4}{20} = \frac{19}{20} = 0.95.$$

However, $\mathbf{1}[s_a = 4] = 0$, so the product in Equation (7) is zero. The lesson fails even though its mean is high. □

The example is a counterexample to the rule “accept whenever the average is high.” It motivates a *noncompensatory* rule: strength in style or completeness cannot cancel a factual weakness.

Worked example A.4. Scores (4, 3, 3, 4, 3) give $S_1 = 17/20 = 0.85$. If $J = 0$ and $b = 1$, every indicator equals 1 and the explanation passes. If one local issue remains, the first factor becomes zero and the same semantic scores fail.

The review makes one initial semantic call. It permits at most one revised call, so

$$N_1 = 1 + \mathbf{1}[\text{first review fails and returns a revision}] \leq 2. \quad (8)$$

This is a direct proof of the call bound: an indicator can only be 0 or 1.

A.4 Gate 2 technical normalization

Define

$$\text{clip}_{[0,1]}(x) = \max(0, \min(1, x)), \quad \rho(x; a, b) = \text{clip}_{[0,1]} \left(\frac{x - a}{b - a} \right), \quad b > a. \quad (9)$$

If $x \leq a$, then $\rho = 0$; if $a < x < b$, it rises linearly; if $x \geq b$, it equals 1. The clipping function therefore converts measurements with different units into comparable normalized components without allowing a very large measurement to exceed 1.

For frame i , let w_i, h_i be dimensions, μ_i mean grayscale brightness, σ_i grayscale contrast, e_i edge variation, and c_i lower-caption-band variation. The code computes

$$r_i = \begin{cases} 1, & w_i \geq 1200 \text{ and } h_i \geq 800, \\ 0.4, & \text{otherwise,} \end{cases} \quad (10)$$

$$x_i = \min\{\rho(\mu_i; 18, 70), \rho(255 - \mu_i; 18, 70)\}, \quad (11)$$

$$q_i = \rho(\sigma_i; 18, 60), \quad (12)$$

$$d_i = \rho(e_i; 14, 46), \quad (13)$$

$$b_i = \rho(c_i; 8, 28), \quad (14)$$

$$t_i = 0.20r_i + 0.20x_i + 0.25q_i + 0.25d_i + 0.10b_i. \quad (15)$$

Each component lies in $[0, 1]$, and the weights sum to 1. Proposition A.1 therefore proves $t_i \in [0, 1]$.

Worked example A.5. For components (1.00, 0.80, 0.70, 0.60, 0.90),

$$t_i = 0.20 + 0.16 + 0.175 + 0.15 + 0.09 = 0.775.$$

The arithmetic is visible and reproducible. It does not prove that the diagram teaches the right idea; that is why a separate semantic audit follows.

A.5 Why every frame must pass

For $n = 7$ frames, define

$$T = \frac{1}{n} \sum_{i=1}^n t_i, \quad P_T = \mathbf{1}[T \geq 0.42] \prod_{i=1}^n \mathbf{1}[t_i \geq 0.42].$$

Proposition A.6 (All-frame equivalence). *The product $\prod_i \mathbf{1}[t_i \geq 0.42]$ equals 1 if and only if every frame meets the technical threshold.*

Proof. If every frame meets the threshold, every indicator is 1 and their product is 1. Conversely, if even one frame fails, its indicator is 0. A product containing a zero is zero. These are the only two cases. $\square$

A mean alone can hide a weak teaching step. If six frames score 0.85 and one scores 0.30, then

$$T = \frac{6(0.85) + 0.30}{7} = \frac{5.40}{7} \approx 0.771.$$

The mean looks strong, but the seventh frame may break the explanation. The product rule catches it.

There is also a probability reason. Suppose, only for illustration, that each frame independently has defect probability p . The probability that no frame is defective is $(1 - p)^7$. Therefore

$$\Pr(\text{at least one defective frame}) = 1 - (1 - p)^7. \quad (16)$$

The proof uses the complement rule: “at least one” is the opposite of “none.”

Per-frame defect probability p	0.01	0.05	0.10	0.20
At least one defect in seven	0.068	0.302	0.522	0.790

Table 5: Illustrative values of $1 - (1 - p)^7$. Independence is an assumption for this calculation, not a measured property of generated frames.

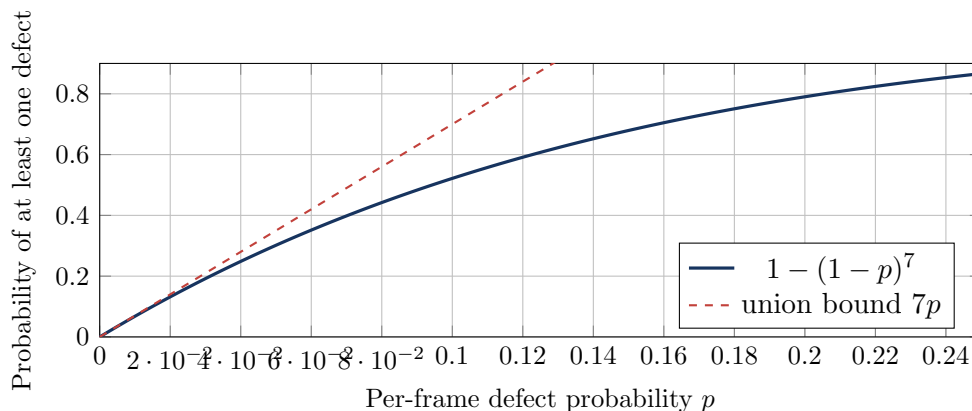


Figure 9: Seven opportunities for failure make a per-frame check important. Without independence, the union bound still gives $\Pr(\text{any defect}) \leq 7p$ when every frame has defect probability at most p .

A.6 Gate 2 semantic score and final rule

Let $v_k \in \{1, 2, 3, 4\}$ score semantic accuracy, teaching alignment, continuity, legibility, and visual load. Then

$$M = \frac{1}{20} \sum_{k=1}^5 v_k, \quad S_2 = 0.35T + 0.65M. \quad (17)$$

Because $T, M \in [0, 1]$, Proposition A.1 proves $S_2 \in [0, 1]$. If v_a is semantic accuracy and b_2 the reviewer’s Boolean result, the exact decision is

$$P_2 = P_T \mathbf{1}[v_a = 4] \mathbf{1}\left[\min_k v_k \geq 3\right] \mathbf{1}[b_2 = 1] \mathbf{1}[S_2 \geq 0.65]. \quad (18)$$

For $T = 0.78$ and scores $(4, 3, 3, 4, 3)$, $M = 17/20 = 0.85$ and

$$S_2 = 0.35(0.78) + 0.65(0.85) = 0.8255.$$

The set passes only if the other indicator rules also equal 1. Again, a combined score summarizes quality but does not replace a mandatory accuracy condition.

A.7 Why a cheap effective gate should come early

The production pipeline has dependency constraints: Gate 2 cannot run before images exist. Within the set of checks that could inspect the same item, elementary algebra gives an ordering rule.

Theorem A.7 (Two-check ordering rule). *Suppose check A costs $c_A > 0$ and rejects a fraction $r_A > 0$, while check B costs $c_B > 0$ and rejects a fraction $r_B > 0$. A rejected item stops. Running A before B has no larger expected checking cost than running B before A exactly when*

$$\frac{c_A}{r_A} \leq \frac{c_B}{r_B}. \quad (19)$$

Proof. With A first, we always pay c_A and pay c_B only for the fraction $1 - r_A$ that continues:

$$E_{A \rightarrow B} = c_A + (1 - r_A)c_B.$$

Similarly,

$$E_{B \rightarrow A} = c_B + (1 - r_B)c_A.$$

Now subtract common terms:

$$\begin{aligned} E_{A \rightarrow B} \leq E_{B \rightarrow A} &\iff c_A + c_B - r_A c_B \leq c_A + c_B - r_B c_A \\ &\iff r_A c_B \geq r_B c_A \\ &\iff \frac{c_A}{r_A} \leq \frac{c_B}{r_B}. \end{aligned}$$

The last step divides by the positive number $r_A r_B$. □

Worked example A.8. A local check costs 0.10 units and rejects 20%, so $c/r = 0.10/0.20 = 0.50$. A model audit costs 1 unit and rejects 30%, so $c/r = 1/0.30 \approx 3.33$. The local check belongs first under the theorem's assumptions. This supports the local-first structure of both gates.

The theorem is not permission to violate dependencies or fairness requirements. It assumes both checks apply to the same incoming item and that rejection behavior remains comparable after reordering.

A.8 Expected cost of the complete cascade

Let C_T include drafting and Gate 1, C_R research, C_I planning plus seven images and Gate 2, and C_V narration plus composition. Let p_1 be the probability Gate 1 passes and p_2 the conditional probability Gate 2 passes. Then

$$\mathbb{E}[C] = C_T + p_1(C_R + C_I) + p_1 p_2 C_V. \quad (20)$$

To derive this, define $I_1 = 1$ when Gate 1 passes and $I_2 = 1$ when both gates pass. The realized cost is

$$C = C_T + I_1(C_R + C_I) + I_2 C_V.$$

Taking averages over many runs gives

$$\mathbb{E}[C] = C_T + \mathbb{E}[I_1](C_R + C_I) + \mathbb{E}[I_2]C_V.$$

For an indicator, its average equals the probability that it is 1. Thus $\mathbb{E}[I_1] = p_1$ and $\mathbb{E}[I_2] = p_1 p_2$, which yields Equation (20).

With illustrative values $C_T = 1$, $C_R = 1$, $C_I = 7$, $C_V = 2$, $p_1 = 0.8$, and $p_2 = 0.9$,

$$\mathbb{E}[C] = 1 + 0.8(1 + 7) + (0.8)(0.9)(2) = 8.84.$$

Always running every stage costs 11 units. The expected saving is $11 - 8.84 = 2.16$ units. Provider prices and pass rates must be measured before turning this example into a budget estimate.

A.9 Why parallel frame diagnostics reduce waiting

Let the seven technical frame checks take times $t_1, \dots, t_7 \geq 0$. Sequential time is

$$T_{\text{seq}} = \sum_{i=1}^7 t_i.$$

With enough workers and no overhead, parallel time would be $T_{\text{par}} = \max_i t_i$.

Proposition A.9 (Basic parallel-time bound). *For nonnegative task times,*

$$\max_i t_i \leq \sum_i t_i.$$

Therefore ideal parallel execution cannot take longer than sequential execution.

Proof. The sum contains the largest term plus other terms that are all nonnegative. It must be at least as large as its largest term. $\square$

The implementation uses at most four workers, so seven equal-duration checks require two waves: four checks, then three. If each takes t , the idealized time falls from $7t$ to $2t$, a speedup of $7/2 = 3.5$. For unequal times and real scheduling, a lower bound is

$$T_{\text{par}} \geq \max \left(\max_i t_i, \frac{\sum_i t_i}{4} \right).$$

Thread overhead and shared resources can reduce the achieved speedup, which is why latency remains an empirical metric.

A.10 Why the scene durations sum to the target

Let n be the number of captions and

$$w_i = \max(5, \text{caption word count}_i), \quad W = \sum_{i=1}^n w_i.$$

If measured narration duration is A , the target is

$$D = \max(A + 0.65, 3.2n).$$

Without measured audio, the fallback estimate is

$$A = \max \left(3.2n, \frac{W}{2.35} + 1.2 \right).$$

The scene allocation is

$$d_i = 3.2 + (D - 3.2n) \frac{w_i}{W}. \tag{21}$$

The maximum in D ensures $D - 3.2n \geq 0$.

Proposition A.10 (Timing conservation). *Every duration in Equation (21) is at least 3.2 seconds and the durations sum exactly to D .*

Proof. Since $D - 3.2n \geq 0$ and $w_i/W > 0$, the added term is nonnegative, so $d_i \geq 3.2$. Also

$$\begin{aligned} \sum_{i=1}^n d_i &= \sum_i 3.2 + (D - 3.2n) \sum_i \frac{w_i}{W} \\ &= 3.2n + (D - 3.2n) \frac{W}{W} \\ &= D. \end{aligned}$$

No time is created or lost by the allocation. □

Worked example A.11. For three caption weights (5, 10, 15) and target $D = 15$ seconds, the floor consumes $3(3.2) = 9.6$ seconds. The remaining 5.4 seconds is divided in ratio $5 : 10 : 15 = 1 : 2 : 3$, adding (0.9, 1.8, 2.7). Durations become (4.1, 5.0, 5.9) seconds and sum to 15.

A.11 Why the displayed media progress never moves backward

Before media generation, the interface uses milestones 0.08 (draft), 0.25 (Gate 1), 0.34 (research when enabled), and 0.40 (prepare visuals). The pipeline reports 0.44 for storyboarding. For frame $i \in \{1, \dots, 7\}$,

$$p_{i,\text{start}} = 0.48 + 0.30 \frac{i - 0.5}{7}, \quad (22)$$

$$p_{i,\text{done}} = 0.48 + 0.30 \frac{i}{7}. \quad (23)$$

Then it reports 0.80 for visual review, 0.86 for preview, 0.90 for narration, 0.96 for composition, and 1.00 for completion.

For a fixed i , $p_{i,\text{done}} - p_{i,\text{start}} = 0.15/7 > 0$. Between frames,

$$p_{i+1,\text{start}} - p_{i,\text{done}} = 0.15/7 > 0.$$

The seventh completion is $0.48 + 0.30 = 0.78 < 0.80$, so the next stage is also larger. This proves monotonicity of the media display. The numbers are stage fractions, not estimates of elapsed or remaining time.

Each stage has a list of N tips. The chosen index is

$$j = (g + i + \sum_{c \in s} \text{ord}(c)) \bmod N, \quad (24)$$

where g is grade, i the frame number or zero, and s the subject text. By the definition of a remainder, $0 \leq j < N$, so the index is always valid. This rule is deterministic and lightweight; it is not a model of the learner.

A.12 Concept graph direction, filtering, and self-report progress

Write the concept neighborhood as a directed labeled graph $G = (V, E)$. The central lesson is v_0 . Each related topic v_i receives a role

$$r(v_i) \in \{\text{prerequisite, application, contrast, next, related}\}.$$

Prerequisite edges point toward v_0 ; application and next-step edges point outward; contrast is drawn as a comparison; related concepts use a neutral connection. Selecting role a displays the subgraph containing v_0 and nodes with $r(v_i) = a$. Filtering therefore removes items from the view but does not create new topics, call a model, or change stored lesson content.

Let $z_i = 1$ when the learner checks “I can explain this link” and $z_i = 0$ otherwise. The displayed fraction is

$$M_G = \frac{1}{n} \sum_{i=1}^n z_i. \quad (25)$$

Because $0 \leq \sum_i z_i \leq n$, dividing by positive n proves $0 \leq M_G \leq 1$. Changing one unchecked item to checked increases the numerator by 1, so progress increases by exactly $1/n$. This is a proof about the bar, not proof of knowledge; z_i is self-report.

A.13 Provider-key readiness as Boolean algebra

Let $K_T = 1$ when a nonempty text key is present, $U_V = 1$ when illustrated video is requested, and $K_I = 1$ when the required media key is present. The Create button uses

$$R = K_T[(1 - U_V) + U_V K_I]. \quad (26)$$

If video is off, $U_V = 0$ and $R = K_T$. If video is on, $U_V = 1$ and $R = K_T K_I$. This gives the truth table:

K_T	U_V	K_I	R	Meaning
0	0	0	0	No written lesson: text key missing
1	0	0	1	Text-only lesson allowed
1	1	0	0	Video requested but media key missing
1	1	1	1	Text and illustrated video allowed

Table 6: Readiness logic. With OpenAI text, the same entered key supplies both K_T and K_I . With DeepSeek text, K_I appears only after the optional media control is enabled.

B Evaluation Mathematics and Counterexamples

B.1 Confusion-matrix measures

For a binary harmful-artifact label, let TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives. Then

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad (27)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (28)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (29)$$

$$\text{F1} = \frac{2 \text{Precision Recall}}{\text{Precision} + \text{Recall}}. \quad (30)$$

Substituting the precision and recall fractions and simplifying gives the equivalent form

$$\text{F1} = \frac{2TP}{2TP + FP + FN}. \quad (31)$$

This makes clear that TN does not affect F1.

If $TP = 36$, $TN = 54$, $FP = 6$, and $FN = 4$, accuracy is 0.90, precision is $36/42 \approx 0.857$, recall is $36/40 = 0.90$, and F1 is approximately 0.878.

B.2 Why accuracy alone can mislead

Suppose 100 artifacts contain 95 acceptable and 5 harmful examples. A useless checker that always predicts “acceptable” gets 95 correct, so accuracy is 95%. Yet it detects zero harmful artifacts: $TP = 0$ and recall is $0/5 = 0$. This counterexample proves that high accuracy does not imply useful harmful-artifact detection on an imbalanced dataset.

A threshold τ can instead be examined using a loss

$$L(\tau) = c_{FN}FN(\tau) + c_{FP}FP(\tau), \quad (32)$$

where c_{FN} and c_{FP} express the relative costs of a missed harmful artifact and an unnecessary block. The code’s current thresholds are engineering settings, not values learned by minimizing a published classroom loss.

AUROC summarizes ranking as the threshold changes [citepfawcett2006roc](#). It does not select the classroom operating point or reveal calibration by itself. For multiclass Gate 1 results, a publication must state whether reported averages are macro, micro, or weighted.

B.3 What these proofs establish—and what they do not

The proofs establish arithmetic properties: bounded scores, monotone responses to component changes, exact Boolean pass logic, conserved duration, valid progress indices, and idealized cost or latency comparisons. They do not establish any of the following:

- that an LLM rubric score equals factual truth;
- that pixel variation equals diagram correctness;
- that frame failures are independent;
- that weights or thresholds are optimal for students;
- that a checked concept is mastered;
- that the lesson improves immediate or delayed learning.

Those claims require teacher labels, learner outcomes, calibrated validation data, and transparent reporting of uncertainty.

C Threshold and Rubric Registry

Rule	Current value	Reason and limitation
Gate 1 word window	70–420	Rejects very thin or unfocused English explanations; not a multilingual reading standard.
Mean sentence length	≤ 27 words	Flags dense prose; does not compute a validated readability grade.
Gate 1 accuracy	$= 4/4$	Treats factual correctness as noncompensatory; still depends on the review model.
Other Gate 1 dimensions	$\geq 3/4$	Requires acceptable completeness, flow, grade fit, and clarity.
Maximum repairs	1	Bounds cost and prevents open-ended self-revision.
Technical frame threshold	0.42	Screens severe render defects before semantic review.
Gate 2 combined threshold	0.65	Engineering acceptance boundary after required component rules.
Gate 2 weights	0.35 / 0.65	Gives semantic meaning more weight than pixel quality; not learned from outcomes.
Minimum scene duration	3.2 s	Provides a reading and transition floor; accessibility testing is still needed.
Narration pace estimate	2.35 words/s	Fallback heuristic used only when measured audio is absent.
Media progress milestones	0.08–1.00	Monotone stage fractions; not a time-remaining prediction.
Concept self-report	m/n	Counts links marked explainable; not an assessment of mastery.
Frame workers	at most 4	Reduces diagnostic wall time; achieved speedup depends on platform.

Table 7: Current thresholds and interface constants reproduced from the implementation.

D Detailed Deployment Checklist

1. Fork or obtain administrator access to the GitHub repository.
2. Confirm that `streamlit.app.py`, `requirements.txt`, and `.python-version` are present.
3. Run the deployment test command from the test protocol in the main text on Python 3.12.
4. Open <https://share.streamlit.io/> and authenticate with the repository-owning GitHub account.
5. Choose repository `allanzhang721/AiLessonStudio`.
6. Choose branch `agent/streamlit-production`.
7. Set entrypoint `streamlit.app.py` and Python 3.12.
8. Do not add a shared provider key unless the deployment owner intends to pay for visitor requests. The interface accepts a visitor-owned text key and reveals an optional OpenAI media key only for a text-only provider.
9. Deploy, inspect build logs, open the public URL, and confirm visibility settings permit the intended audience.
10. Test both providers: OpenAI should expose text, research, image model, and voice controls; DeepSeek should show a text-only notice, keep media off by default, and reveal the optional OpenAI image-key path only after the learner enables it.
11. Generate a text-only lesson, an illustrated lesson, a failed-key request, a Gate 1 repair, and a visual regeneration.
12. Record the Git commit, date, provider models, and any calibrated thresholds with published results. Official Streamlit guidance recommends declaring dependencies, selecting the intended Python version, and keeping secrets outside the repository (Streamlit, 2026a,b). Repository credentials are required to configure the hosted app; a source-code commit alone cannot change an account’s cloud visibility.

E Paper Reproduction and Version Statement

Compile the paper from the repository root with:

```
cd docs/whitepaper
pdflatex Explain_It_White_Paper.tex
bibtex Explain_It_White_Paper
pdflatex Explain_It_White_Paper.tex
pdflatex Explain_It_White_Paper.tex
```

The compilation expects the PNG experiment figures in `../..paper_figures/`. All architecture and process diagrams are embedded as TikZ source in this file. The output should be archived together with the source, bibliography, figure files, software commit, and a short change log.

This white paper describes the code state through production commit `16bfde7` in August 2026. Generative AI assisted with organization, editing, and diagram planning. The authors remain responsible for checking the implementation claims, citations, equations, and publication permissions. The uploaded SLaTR manuscript was consulted only for general writing habits and exposition structure. It is marked as an anonymous submission whose sharing is prohibited, so it is neither cited as technical evidence nor reproduced.

References

- Chloe Autio, Reva Schwartz, Jesse Dunietz, Shomik Jain, Martin Stanley, Elham Tabassi, Kamie Roberts, et al. Artificial intelligence risk management framework: Generative artificial intelligence profile. Technical Report NIST AI 600-1, National Institute of Standards and Technology, 2024. URL <https://doi.org/10.6028/NIST.AI.600-1>.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019. doi: 10.18653/v1/N19-1423. URL <https://aclanthology.org/N19-1423/>.
- Tom Fawcett. An introduction to roc analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006. doi: 10.1016/j.patrec.2005.10.010. URL <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Logan Fiorella and Richard E. Mayer. Principles for reducing extraneous processing in multimedia learning. In Richard E. Mayer and Logan Fiorella, editors, *The Cambridge Handbook of Multimedia Learning*. Cambridge University Press, 3 edition, 2021. URL <https://www.cambridge.org/core/books/cambridge-handbook-of-multimedia-learning/>.
- John Hattie and Helen Timperley. The power of feedback. *Review of Educational Research*, 77(1):81–112, 2007. doi: 10.3102/003465430298487. URL <https://doi.org/10.3102/003465430298487>.
- Micah Hodosh, Peter Young, and Julia Hockenmaier. Framing image description as a ranking task: Data, models and evaluation metrics. *Journal of Artificial Intelligence Research*, 47:853–899, 2013. doi: 10.1613/jair.3994. URL <https://doi.org/10.1613/jair.3994>.
- Enkelejda Kasneci, Kathrin Sessler, Stefan Kuechemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Guennemann, Eyke Huellermeier, et al. Chatgpt for good? on opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103:102274, 2023. doi: 10.1016/j.lindif.2023.102274. URL <https://doi.org/10.1016/j.lindif.2023.102274>.

- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019. doi: 10.48550/arXiv.1907.11692. URL <https://arxiv.org/abs/1907.11692>.
- Richard E. Mayer. *Multimedia Learning*. Cambridge University Press, 2 edition, 2009. doi: 10.1017/CBO9780511811678. URL <https://doi.org/10.1017/CBO9780511811678>.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, pages 8748–8763, 2021. URL <https://proceedings.mlr.press/v139/radford21a.html>.
- Henry L. Roediger and Jeffrey D. Karpicke. Test-enhanced learning: Taking memory tests improves long-term retention. *Psychological Science*, 17(3):249–255, 2006. doi: 10.1111/j.1467-9280.2006.01693.x. URL <https://doi.org/10.1111/j.1467-9280.2006.01693.x>.
- Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019. doi: 10.48550/arXiv.1910.01108. URL <https://arxiv.org/abs/1910.01108>.
- Streamlit. Deploy your app on community cloud. <https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/deploy>, 2026a. Accessed August 5, 2026.
- Streamlit. Secrets management for community cloud. <https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/secrets-management>, 2026b. Accessed August 5, 2026.
- John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988. doi: 10.1207/s15516709cog1202_4. URL https://doi.org/10.1207/s15516709cog1202_4.